



React Essentials

Deep Dive

A clear, example-driven guide to advanced React component patterns

[Fragments](#) · [Props](#) · [Slots](#) · [Events](#) · [Defaults](#)

#	Topic
1	Fragment in Function Components
2	Forwarding Props with Spread Operator
3	Multiple Component Slots
4	Element Identifier as Props (Dynamic Tags)
5	Functions as Props for HTML Events
6	Default Prop Values

1 Fragment in Function Components

What is a Fragment?

React requires every component to return a **single root element**. Fragments let you group multiple elements without adding an extra DOM node (like an unnecessary `<div>`). This keeps your HTML clean and avoids layout / styling side-effects.

Two Ways to Write Fragments

Option A – Import Fragment explicitly:

```
import { Fragment } from 'react';

function UserCard() {

  return (

    <Fragment>

    <h2>Alice</h2>

    <p>Frontend Developer</p>

    </Fragment>

  );

}
```

Option B – Short-hand syntax (most common):

```
function UserCard() {

  return (

    <>

    <h2>Alice</h2>

    <p>Frontend Developer</p>

    </>

  );

}
```

Tip: Use (not <>) when you need to attach a key prop inside a list — short-hand fragments cannot carry attributes.

2 Forwarding Props with Spread Operator

Why Forward Props?

When you wrap a native HTML element (e.g. `<input>`, `<button>`) inside a custom component, you usually want callers to be able to pass any standard HTML attribute. Destructuring known props and spreading the rest (**`...props`**) onto the element achieves this elegantly.

Example – Custom Input Component

```
// Input.jsx

export default function Input({ label, ...props }) {

  return (

    <div className='input-wrapper'>

      <label>{label}</label>

      <input {...props} /> {/* forward: type, placeholder, onChange ... */}

    </div>

  );

}

// Usage

<Input

  label='Email'

  type='email'

  placeholder='you@example.com'

  onChange={handleChange}

/>
```

Tip: Always destructure props you consume explicitly first, then spread the rest. This prevents passing component-only props down to the DOM element.

3 Multiple Component Slots

The Slot Pattern

React doesn't have a built-in 'slot' system (like Web Components), but you can achieve the same result by passing JSX as **named props**. This lets parent components inject different pieces of UI into specific regions of a child layout component.

Example – Card with Header / Body / Footer Slots

```
// Card.jsx

export default function Card({ header, children, footer }) {

  return (

    <div className='card'>

      <div className='card-header'>{header}</div>

      <div className='card-body'>{children}</div>

      <div className='card-footer'>{footer}</div>

    </div>

  );

}

// Usage

<Card

  header={<h2>User Profile</h2>}

  footer={<button>Save</button>}

  >

  <p>Main content goes here via children.</p>

</Card>
```

Tip: children is the default slot. Use extra named props (header, footer, sidebar...) for additional injection points.

4 Element Identifier as Props (Dynamic Tags)

Dynamic Component Type

You can pass a **component type** or **HTML tag name** as a prop so the receiving component renders the caller's chosen element. Capitalise the variable name so React treats it as a component, not a string.

Example – Section with Configurable Heading

```
// Section.jsx

export default function Section({

  title,

  children,

  titleComponent: TitleComponent = 'h2', // default: h2

}) {

  return (

    <section>

      <TitleComponent>{title}</TitleComponent>

      {children}

    </section>

  );

}

// Usage – native tag

<Section title='Overview' titleComponent='h1'>...</Section>

// Usage – custom component

<Section title='Overview' titleComponent={StyledHeading}>...</Section>
```

Tip: Always rename the prop to a PascalCase variable (e.g. TitleComponent) before using it in JSX so React knows it is a component reference.

5 Functions as Props for HTML Events

Passing Callbacks

React uses camelCase event handler props (onClick, onChange, onSubmit...). You can pass any function — defined inline or externally — as a prop. This is the primary way child components communicate with their parents ('lifting state up').

Example – Button with onClick Prop

```
// Button.jsx

export default function Button({ label, onClick, type = 'button' }) {

  return (

    <button type={type} onClick={onClick}>

      {label}

    </button>

  );

}


// Parent.jsx

function Parent() {

  function handleSave() {

    console.log('Saved!');

  }

  return <Button label='Save' onClick={handleSave} />;

}
```

Passing Data Back to the Parent

```
// searchBar.jsx

export default function SearchBar({ onSearch }) {

  return (

    <input

      type='search'

      onChange={(e) => onSearch(e.target.value)}

    />

  );

}
```

```
// Usage
```

```
<SearchBar onSearch={(query) => console.log('Searching:', query)} />
```

Tip: Pass the function reference (`onClick={handleSave}`), not its result (`onClick={handleSave()}`). Adding `()` calls it immediately on render.

6 Default Prop Values

Why Use Defaults?

Default values make components resilient — they work correctly even when a caller omits optional props. In modern React you set defaults directly in the **destructuring assignment** of the function signature.

Example – Avatar with Sensible Defaults

```
// Avatar.jsx

export default function Avatar({

  src = '/default-avatar.png',

  alt = 'User avatar',

  size = 48,

  shape = 'circle', // 'circle' | 'square'

}) {

  const style = {

    width: size,

    height: size,

    borderRadius: shape === 'circle' ? '50%' : '8px',

  };

  return <img src={src} alt={alt} style={style} />;

}

// Minimal usage – all defaults apply

<Avatar />

// Partial override
```

```
<Avatar src='/alice.jpg' size={64} />
```

Tip: Prefer destructuring defaults over the old defaultProps object — it is cleaner, co-located with the props, and works for both JS and TypeScript.

Code Examples — GitHub References

Each example below is taken from the official course repository. Click the link or open it in your browser to explore the full source.

Example 01

Fragment in Function Component

Demonstrates how to use `React.Fragment` (and the shorthand `<>`) to return multiple sibling elements from a component without adding an extra wrapper node to the real DOM.

<https://github.com/academind/react-complete-guide-course-resources/tree/main/code/04%20Essentials%20Deep%20Dive/02-fragments>

```
// App.jsx (simplified)

import { Fragment } from 'react';

export default function App() {

  return (

    <> {/* shorthand fragment */}

    <Header />

    <main>

    <CoreConcepts />

    <Examples />

    </main>

    </>

  );

}
```

Example 02

Splitting Components — Multiple Component Slots

Shows how to divide a large component into smaller focused ones and how to use the children prop plus additional named JSX props to create flexible multi-slot layouts.

<https://github.com/academind/react-complete-guide-course-resources/tree/main/code/04%20Essentials%20Deep%20Dive/03-splitting-components>

```
// Section.jsx

export default function Section({ title, children }) {

  return (

    <section>

      <h2>{title}</h2>

      {children}

    </section>

  );

}

// Usage

<Section title='Core Concepts'>

  <CoreConceptsList />

</Section>
```

Example 03

Forwarding Props — Spread Operator in Function Components

Illustrates how to capture all 'extra' props with the rest/spread syntax (...props) and pass them straight through to the underlying HTML element, keeping wrapper components transparent.

<https://github.com/academind/react-complete-guide-course-resources/tree/main/code/04%20Essentials%20Deep%20Dive/04-forwarding-props>

```
// Input.jsx

export default function Input({ label, id, ...props }) {

  return (

    <p>

      <label htmlFor={id}>{label}</label>

      <input id={id} {...props} /> {/* type, placeholder, onChange, ... */}

    </p>

  );

}

// Usage

<Input label='Name' id='name' type='text' placeholder='Your name' />
```

Example 04

Multiple JSX Slots — Passing Functions as Event Props

Covers the pattern of accepting a function as a prop and wiring it to a native HTML event handler — the standard React way to let parents react to events emitted by children.

<https://github.com/academind/react-complete-guide-course-resources/tree/main/code/04%20Essentials%20Deep%20Dive/05-multiple-jsx-slots>

```
// TabButton.jsx

export default function TabButton({ children, onSelect }) {

  return (

    <li>

      <button onClick={onSelect}>{children}</button>

    </li>

  );

}

// Usage

<TabButton onSelect={() => handleSelect('components')}>

  Components

</TabButton>
```

Example 05

Dynamic Component Types — Tag Name as Prop + Default Prop Values

Combines two concepts: passing a tag name or component constructor via a prop so the child renders the caller-chosen element, and providing safe default values directly in the destructuring signature.

<https://github.com/academind/react-complete-guide-course-resources/tree/main/code/04%20Essentials%20Deep%20Dive/06-dynamic-component-types>

```
// Section.jsx

export default function Section({
  title,
  children,
  id,
  titleComponent: TitleComponent = 'h2', // default tag
}) {
  return (
    <section id={id}>
      <TitleComponent>{title}</TitleComponent>
      {children}
    </section>
  );
}

// As h1
<Section title='Welcome' titleComponent='h1'>...</Section>

// As custom component
<Section title='Features' titleComponent={AnimatedHeading}>...</Section>
```

Quick Reference — All Concepts

Concept	Key Syntax	Benefit
Fragment	<> ... or	No extra DOM nodes

Forward Props	<code>{ label, ...props }</code>	Transparent wrappers
Multiple Slots	<code>header={} + children</code>	Flexible layouts
Dynamic Tag	<code>titleComponent: T = 'h2'</code>	Reusable tag-agnostic UI
Function Prop	<code>onClick={fn}</code>	Parent-child communication
Default Values	<code>{ size = 48 }</code>	Safe optional props

Based on the Academind 'React - The Complete Guide' course · Section 04 Essentials Deep Dive